

OptArrow Requirements Specification

System Context & Overview

The OptArrow optimization integration engine serves as a bridge connecting various optimization clients to solver backends, utilizing a stable and high-performance transport layer. The system's architecture capitalizes on both Python and Julia backends and provides interfacing capabilities with environments such as MATLAB through a lightweight client interface. The core system logic is designed to handle and solve optimization problems, specifically linear programming (LP) and quadratic programming (QP), by routing inputs from different environments through the appropriate computational pathways and returning results in the caller's originating programming language. The architecture will leverage Apache Arrow for in-memory data transfer due to its demonstrated cross-language compatibility, an essential factor given the diverse programming environments integrated with OptArrow.

Stakeholder Needs & Feedback Mechanisms

User feedback plays a crucial role in refining OptArrow's capabilities to better meet the evolving needs of its stakeholders, including Software Engineers, Researchers, Data Scientists, and Optimization Specialists. The system shall continue to utilize GitHub Issues as the primary mechanism for collecting stakeholder feedback. This approach streamlines communication and allows for easy tracking of user suggestions, issues, and enhancements. However, future system iterations are expected to incorporate more formalized stakeholder feedback review processes and requirement traceability mechanisms, ensuring thorough alignment between user needs and system developments.

Functional Requirements

Input Specifications

- The system shall accommodate any mathematical problem that requires optimization, capturing the input problem definitions efficiently.
- The input format must be compatible with both Python and Julia environments, ensuring seamless data ingestion regardless of the client's development platform.

Behavioral Pathways

- Upon receiving an optimization problem, the system will identify the source environment (e.g., Python or Julia) and route the problem through the most suitable solver backend.
- OptArrow shall employ Apache Arrow for efficient, in-memory interlanguage communication, ensuring that data integrity and speed are maintained across transitions.

Output Specifications

- OptArrow will ensure that the calculated optimization results are delivered back to the client in the same programming language as the input, preserving coding integrity and

facilitating ease of integration into existing workflows.

Non-Functional Requirements

Performance Metrics

- The system shall prioritize high-speed data transfers and computation efficiency, leveraging Apache Arrow's in-memory capabilities to minimize latency in cross-language operations.
- Future iterations shall aim to rigorously benchmark the system's performance, targeting industry-leading optimization problem-solving times.

Compatibility Across Environments

- Broad compatibility shall be maintained across the primary environments (Python, Julia, and MATLAB). This includes ensuring that updates in programming languages' APIs do not break the existing OptArrow integration.

Requirements Traceability & Governance

Formal Traceability Mechanisms

- OptArrow shall establish a clear framework for mapping stakeholder feedback directly to system requirements and subsequent implementations. This will involve developing dedicated tools and processes for tracing requirements from inception through to deployment.

Governance and Compliance

- Currently, the decision-making process is largely guided by direction from the principal investigator. However, future system enhancements will pursue more structured, documented decision-making processes, possibly expanding into formal Architectural Decision Records (ADRs) for enhanced transparency and accountability.

Process Maturity and Gaps

- The current informal architectural evolution necessitates future work on establishing a documented architecture roadmap to guide development. Additional efforts will be made to fill process gaps in architectural governance and decision rationalization, ensuring OptArrow evolves into a mature project with well-defined engineering and governance practices.

These requirements are intended to guide the development and future enhancements of OptArrow, ensuring that the system remains resilient, adaptable, and efficient in meeting the complex needs of optimization problem-solving across multiple programming environments.